

Failure modes → which pattern

Problem	Pattern (R4j)
Slow call, no upper bound	TimeLimiter + client connect/read timeouts
Transient error (blip, 503, 429)	Retry (backoff + jitter, idempotent only)
Dependency down, cascading failure	CircuitBreaker (fail fast)
One dependency eats all threads	Bulkhead / ThreadPoolBulkhead
Quota (yours or theirs), overload	RateLimiter (in-process)
Still need an answer	Fallback : cached, default, empty, fail fast
Repeated identical reads	Cache (JCache) · tail latency: Hedge

Also: retry storm (layers multiply: 3x3x3 = 27 calls), thundering herd (use jitter), resource exhaustion, partial failure. **Idempotency** is the precondition for retries. Fail fast vs degrade gracefully. Hystrix (maintenance since 2018) → Resilience4j.

Dependencies

Boot 4.1 (starter + AspectJ + Actuator; the BOM does **not** manage the Boot 4 starter, so pin it):

```
io.github.resilience4j:resilience4j-bom:2.4.0 (import)
io.github.resilience4j:resilience4j-spring-boot4:2.4.0
org.springframework.boot:spring-boot-starter-aspectj
org.springframework.boot:spring-boot-starter-actuator
```

Boot 3.5: resilience4j-spring-boot3 + spring-boot-starter-aop + actuator. Wrong starter for the Boot major fails at startup.

No AspectJ = registries and endpoints work but @CircuitBreaker etc. are a **silent no-op**. Core modules arrive transitively via micrometer/reactor; list them if unsure.

Add-ons: -reactor (Mono/Flux), -rxjava3, -kotlin, -feign, -micrometer, -all (Decorators), -cache (JCache), -hedge (not in BOM or -all). Dropped in 2.4.0: spring-boot2, spring (5), spring-cloud2. Boot does not manage the R4j version.

CircuitBreaker

Properties (Boot key = builder name) and **defaults**:

- failureRateThreshold **50 %**, slowCallRateThreshold **100 %**, slowCallDurationThreshold **60 s**
- slidingWindowType **COUNT_BASED** (N calls) or **TIME_BASED** (last N seconds); slidingWindowSize **100**
- minimumNumberOfCalls **100** (capped to the window size)
- waitDurationInOpenState **60 s**; automaticTransitionFromOpenToHalfOpenEnabled **false** (the next call moves it)
- permittedNumberOfCallsInHalfOpenState **10**; maxWaitDurationInHalfOpenState **0** (wait for probes forever)
- recordExceptions all / ignoreExceptions none — ignore wins; recordResultPredicate
- 2.4.0: initialState **CLOSED**, transitionToStateAfterWaitDuration

Exception: CaLNotPermittedException (open). Failure and slow calls both count. Needs minimumNumberOfCalls first. Java-only: slidingWindowSynchronizationStrategy, clock, ignoreExceptionsPrecedenceEnabled (customizer). Boot-only: registerHealthIndicator (false), allowHealthIndicatorToFail, eventConsumerBufferSize (100), backoff keys for the open wait.

Retry · RateLimiter · Bulkhead · TimeLimiter

Retry (@Retry): maxAttempts **3** (includes the first call), waitDuration **500 ms**, retryExceptions, ignoreExceptions (ignore wins), retryOnResult, failAfterMaxAttempts false. Throws the last exception, or MaxRetriesExceededException (result retry only).

RateLimiter: limitForPeriod **50**, limitRefreshPeriod **500 ns**, timeoutDuration **5 s** — per cycle, per instance, in-process (AtomicRateLimiter). Throws RequestNotPermitted (→ HTTP 429 + Retry-After).

Bulkhead (semaphore): maxConcurrentCalls **25**, maxWaitDuration **0**; fairCallHandlingEnabled is Java-only.

ThreadPoolBulkhead: coreThreadPoolSize **max(CPU-1,1)**, maxThreadPoolSize **CPU**, queueCapacity **100**, keepAliveDuration **20 ms**, contextPropagators; returns CompletionStage. Both throw BulkheadFullException.

TimeLimiter: timeoutDuration **1 s**, cancelRunningFuture **true**. @TimeLimiter needs a CompletionStage, Mono/Flux or RxJava return type (ILlegalReturnTypeErrorException); throws TimeoutException.

Backoff: wait(n) = waitDuration × multiplierⁿ⁻¹; jitter = x random in [1-f, 1+f]; cap applied after jitter. Boot keys: enableExponentialBackoff, exponentialBackoffMultiplier (default **1.5**), exponentialMaxWaitDuration, enableRandomizedWait, randomizedWaitFactor (0.5); they need waitDuration in the same block. intervalBiFunction overrides them (honour Retry-After).

Rule: retry only idempotent calls (GET, PUT, DELETE, keyed POST), only transient errors (IO, 502/503/504, 429), never 4xx. No built-in retry budget. Default Retry retries CaLNotPermittedException and BulkheadFullException: ignore them.

Limiter notes: timeoutDuration parks the caller; @RateLimiter(permits) is **ignored** by the Spring aspect (1 permit per call); change at runtime with changeLimitForPeriod.

Testing tips

- Registry: transitionToOpenState(), transitionToForcedOpenState(), reset(), getMetrics().
- Time: custom clock and currentTimestampFunction; test profile with window 4, waits 100 ms, no jitter.
- @SpringBootTest: AopUtils.isAopProxy, fallback invoked, order via events.
- WireMock: withFixedDelay, 5xx, 429 + Retry-After, faults; Toxiproxy (Testcontainers): latency, resets.
- Reactor: TestVerifier; assert on /actuator/circuitbreakers and meters.

Anti-patterns

- One instance shared by several backends — one per backend.
- Retrying non-idempotent calls; retries at every layer (retry amplification).
- A breaker on every call; a breaker with no timeout; inner timeout longer than the caller's.
- Fallbacks that hide the outage (a Throwable fallback also swallows BulkheadFullException).
- Breaker health wired to liveness; default thresholds copied without traffic data (p99 → slowCallDurationThreshold).
- Forgetting spring.http.clients.connect-timeout/read-timeout; relying on an annotation without AspectJ.
- Runbook: FORCE_OPEN during maintenance; initialState: FORCED_OPEN for persistence.

Alternatives: Resilience4j vs Bucket4j

	Resilience4j	Bucket4j 8.21
Scope	outbound calls	in- and outbound
Algorithm	fixed cycles	token bucket
Distributed	no: per JVM	yes: shared store
Per-key	no	yes: per key
Other patterns	CB, Retry...	none

Use both → **Spring Boot Bucket4j Cheat Sheet**

application.yml skeleton & customizers

```
# modules: circuitbreaker, retry, ratelimiter, # bulkhead, thread-pool-bulkhead, timelimiter resilience4j: circuitbreaker: configs: default: {...} strict: {...} instances: pricing: baseConfig: strict slidingWindowSize: 20 waitDurationInOpenState: 20s
```

- Unlisted names get configs.default. Durations: 500ms, 2s, PT2S; a bare number = ms.
- Exceptions/predicates: fully qualified class names (public no-arg ctor). A misspelt class fails startup; unknown keys are silently ignored.
- baseConfig: CircuitBreaker replaces default; the other modules always root at default. Self-reference fails.
- Customizers (7 types, e.g. CircuitBreakerConfigCustomizer.of("pricing", b → ...)) run last and beat YAML and the command line; the place for Clock and Java-only settings.
- Profiles: a profile that changes default does not reach an instance that sets the same key. SpEL names only for #/@ strings. @RefreshScope registries exist in the Boot 4 starter only.
- resilience4j.<module>.tags.<k> and .metrics.enabled; *AspectOrder keys live under the module prefix.

Annotations, fallbacks & aspect order

@CircuitBreaker @Retry @RateLimiter @Bulkhead(type=SEMAPHORE|THREADPOOL) @TimeLimiter @Timer — attributes name, fallbackMethod, configuration (2.4.0), method or class level.

Return types: sync for all but @TimeLimiter/THREADPOOL; CompletionStage for those; Mono/Flux need resilience4j-reactor (else silent no-op); Future is treated as sync.

Fallback signature: same class, same parameters + **a trailing exception** parameter, return type assignable; closest exception type wins; an exception-only method matches any annotated method of the class; ambiguous overloads fail per call. Put it on the **outermost** annotation: a fallback on the inner @CircuitBreaker makes @Retry see success.

Default order (lower number = outer):

```
Retry(-5) > CircuitBreaker(-4) > RateLimiter(-3) > TimeLimiter(-2) > Bulkhead(-1) > method = Retry(CB(RL(TL(Bulkhead(fn)))) (@Timer, @Transactional: LOWEST_PRECEDENCE)
```

Change with resilience4j.<module>. <module>AspectOrder (bulkheadAspectOrder settable since 2.4.0). Retry outside CB inflates breaker metrics by retries (wanted: failure rate counts attempts).

AOP pitfalls: self-invocation bypasses the proxy; final method → NPE, final class fails, private ignored; annotation on an interface method is ignored; @Transactional + @Retry = one transaction per attempt by default.

Programmatic · reactive · cache · hedge

- Programmatic:** inject CircuitBreakerRegistry etc.; cb.executeSupplier(...). Decorators.ofSupplier(fn).withCircuitBreaker(cb).withRetry(r).withFallback(f): **last added = outermost**. Decorate RestClient (interceptor), WebClient (filter), @HttpExchange clients; count 5xx/429 with recordResultPredicate.
- Reactor:** mono.transformDeferred(CircuitBreakerOperator.of(cb)); last applied = outermost; BulkheadOperator blocks the subscriber. RxJava: compose. Kotlin: executeSuspendFunction; annotations do not protect suspend functions.
- Cache** (JCache): Decorators.withCache; null never cached. **Hedge:** second request after a delay, first completion wins.

Observability

Actuator (expose them): circuitbreakers, circuitbreakerevents, retries, retryevents, ratelimiters, ratelimiterevents, bulkheads, bulkheadevents, timelimiters, timelimiterevents, timers, timerevents.

Write op: POST /actuator/circuitbreakers/{name} body {"updateState": "CLOSE|FORCE_OPEN|DISABLE"} (per JVM, in memory). Secure it or set management.endpoint.circuitbreakers.access: read-only.

Health: needs management.health.circuitbreakers.enabled=true (and .ratelimiters) + per-instance registerHealthIndicator. CLOSED → UP, OPEN → CIRCUIT_OPEN, HALF_OPEN → CIRCUIT_HALF_OPEN (top level stays UP); DOWN only with allowHealthIndicatorToFail. **Keep breaker health out of the Kubernetes liveness probe.**

Meters (Prometheus: dots → underscores): resilience4j.circuitbreaker.state / .calls / .failure.rate / .slow.call.rate / .not.permitted.calls / .buffered.calls; resilience4j.retry.calls; resilience4j.ratelimiter.available.permissions / .waiting_threads; resilience4j.bulkhead.available.concurrent.calls / .queue.depth; resilience4j.timelimiter.calls.

Grafana: repo grafana_dashboard.json (needs the application tag; covers CB, retry, semaphore bulkhead only). Alert on: breaker OPEN, retry rate, bulkhead saturation, limiter waiting threads. Log with getEventPublisher().onEvent.

Spring Cloud CB · Gateway · OpenFeign

- SCCB:** CircuitBreakerFactory.create("id").run (supplier, fallback) (also ReactiveCircuitBreakerFactory); configure with Customizer<Resilience4JCircuitBreakerFactory>. **TimeLimiter on by default (1 s)**; thread-pool bulkhead by default (enable-semaphore-default-bulkhead), whose fallback may see ExecutionException.
- Align versions:** SCCB 5.0.3 pins R4j 2.3.0 + -spring-boot3. Import resilience4j-bom *before* spring-cloud-dependencies, exclude -spring-boot3, add -spring-boot4.
- Gateway filter** CircuitBreaker: name, fallbackUri: forward/fallback, statusCodes, resumeWithoutError; open → 503, timeout → 504. RequestRateLimiter (Redis, WebFlux) is not R4j.
- OpenFeign:** spring.cloud.openfeign.circuitbreaker.enabled=true, fallback / fallbackFactory, alphanumeric-ids; plain Resilience4jFeign + FeignDecorators.
- Framework **Retry** implementation: starter spring-cloud-starter-circuitbreaker-framework-retry.

Spring Framework 7 vs Resilience4j

	Spring 7 native	Resilience4j
Retry	@Retryable	Retry
Concurrency	@ConcurrencyLimit	Bulkhead
Breaker, rate, time limiter	none	yes
Metrics, Actuator, health	none (MethodRetryEvent)	yes

Needs @EnableResilientMethods (else silent no-op). @Retryable: maxRetries 3 (= 4 invocations), delay 1000 ms, multiplier 1.0, jitter 0, maxDelay unlimited, includes/excludes, timeout. @ConcurrencyLimit(n): policy BLOCK (default, no timeout) or REJECT (InvocationRejectedException). Native @Retryable is outer to @CircuitBreaker; native + R4j @Retry = 9 calls. Programmatic: RetryTemplate, RetryPolicy. **Spring Retry is archived** → migrate. Use Spring 7 for plain retry/limit; R4j for breaker, rate limit, observability.